STEVEN LOAÍCIGA SEGURA

5 DE NOVIEMBRE DE 2026



TEMA ANTERIOR: GRAMÁTICAS LL(1)

COMPILADORES E INTÉRPRETES

CONTENIDO

1. Propiedades de parsers LL(1)
2. Parsers LL(2)
3. Parsers LL(k)
4. Descenso recursivo
5. Errores y LL(1)
6. Mensajes de error

Propiedades de parsers LL(1)



Sea G una CFG, si G es LL(1), entonces:

1. Si una hilera no se rechaza, se garantiza que obtenemos la derivación más izquierda correcta que la genera.
2. G no es ambigua
3. La complejidad temporal y espacial del parsing LL(1) es lineal con respecto al largo de la hilera



Los parsers LL(1) en cuestión

Propiedades de parsers LL(1)



Torres en sus 30 leyendo el periódico mientras le compila su programa

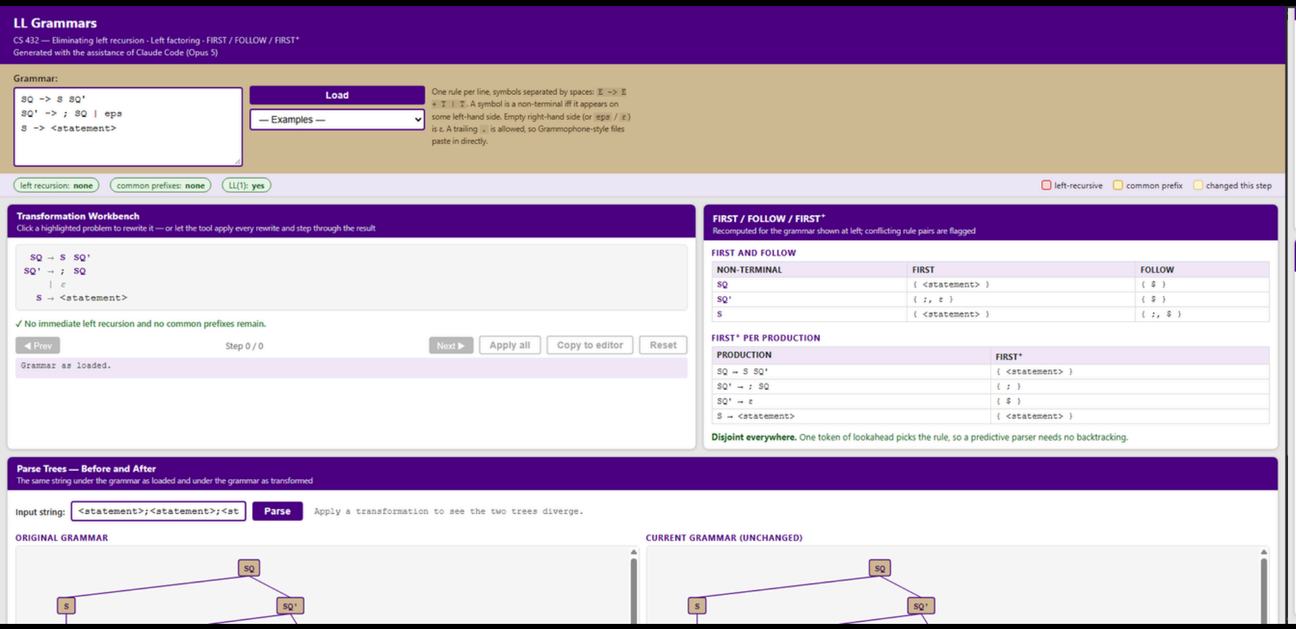
- En general, todos los parsers son lineales; sin embargo, hay unos más complejos
- Bison (la herramienta usada en el proyecto programado 2) genera parsers LAL(1)
- Hay herramientas que generan parsers LL(1), fácilmente encontrables como LLgen
- Que los compiladores fuesen tan veloces le abrió la industria al software

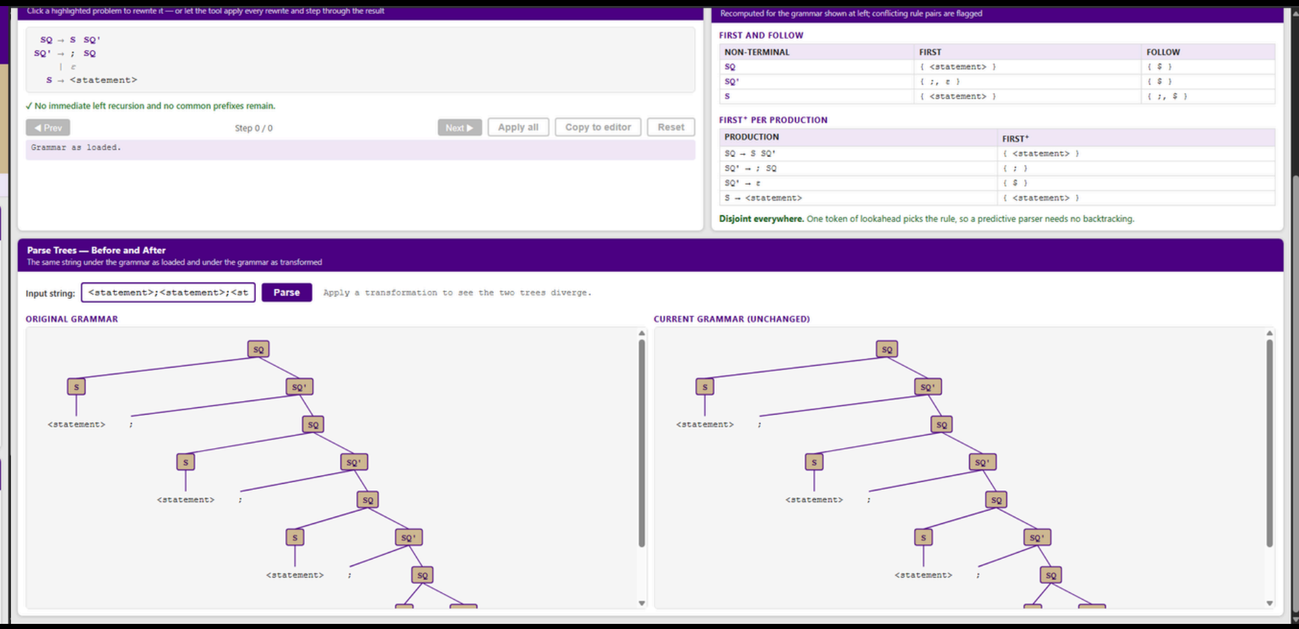


El judío que lo contrató enojado porque piensa que no está haciendo nada

Propiedades de parsers LL(1)

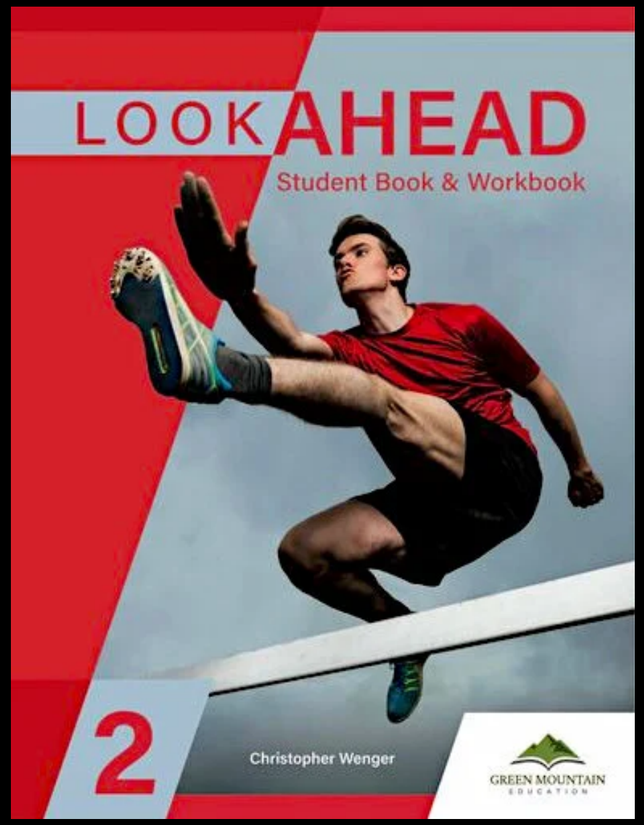
Ejemplo de LLgenerator:





Link

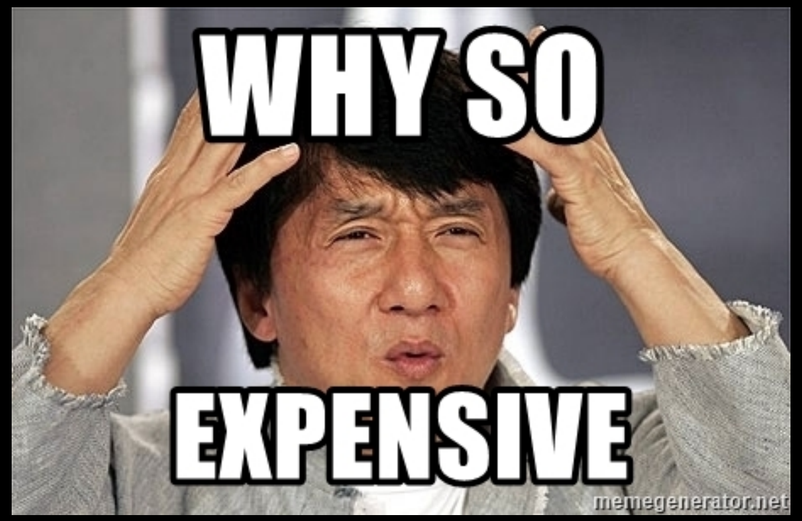
Parsers LL(2)



- Un parser LL(1) puede predecir la regla que debe usarse con solo un símbolo de lookahead
- Hay gramáticas que no son LL(1), aunque no sean recursivas por la izquierda y estén factorizadas
- Puede ser que viendo 2 símbolos por adelantado se pueda predecir la regla que deba usarse
- Parsing LL(2)



Parsers LL(2)



- Se definen las funciones:
  - $FIRST_2()$
  - $FOLLOW_2()$
  - $PREDICT_2()$
- Los cálculos son más costosos
- La tabla de LL(2) tiene $(n^2 + n)$ columnas, donde n es la cantidad de terminales de la gramática.



Parsers LL(2)



TABLA DE PARSING LL(2)

Para la gramática:

1. S → (S) S
2. S → ε

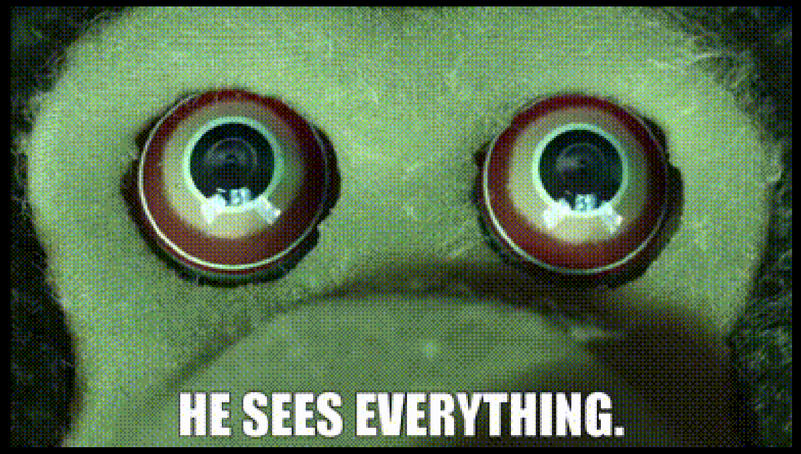
La tabla luce algo así:

	((	()	($	)(	))
S					



Como podemos notar, es algo extensa

Parsers LL(k)

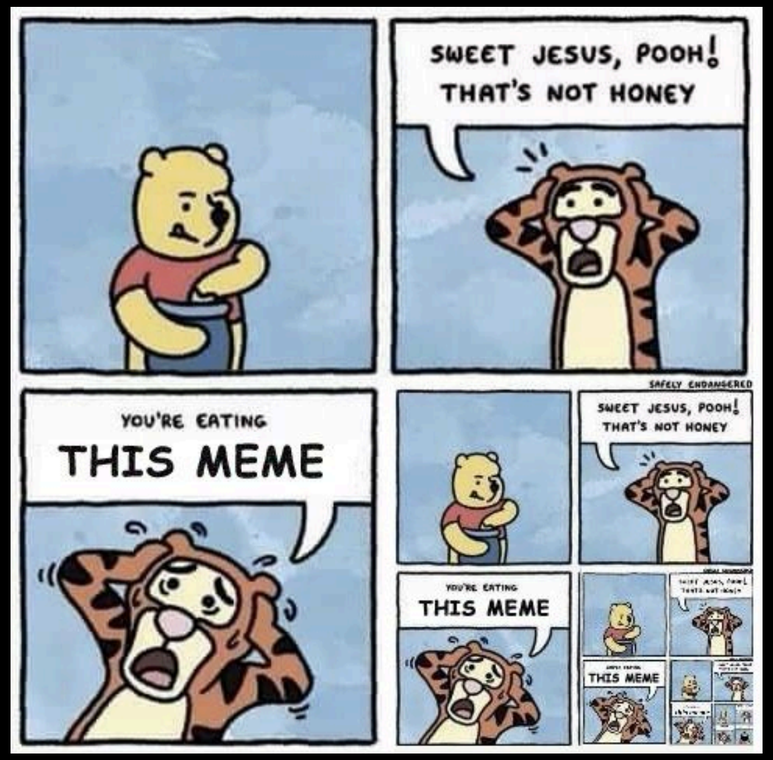


- Caso general: Parsers LL(k)
- Un lookahead de k símbolos para decidir qué regla usar
- Hay:
  - $FIRST_k()$
  - $FOLLOW_k()$
  - $PREDICT_k()$
- Tablas con $(n^k + n^{k-1})$ columnas, donde n es la cantidad de terminales de la gramática
- Casi todos los lenguajes de programación son LL(1) o, si no, se pueden usar otro tipo de parsers

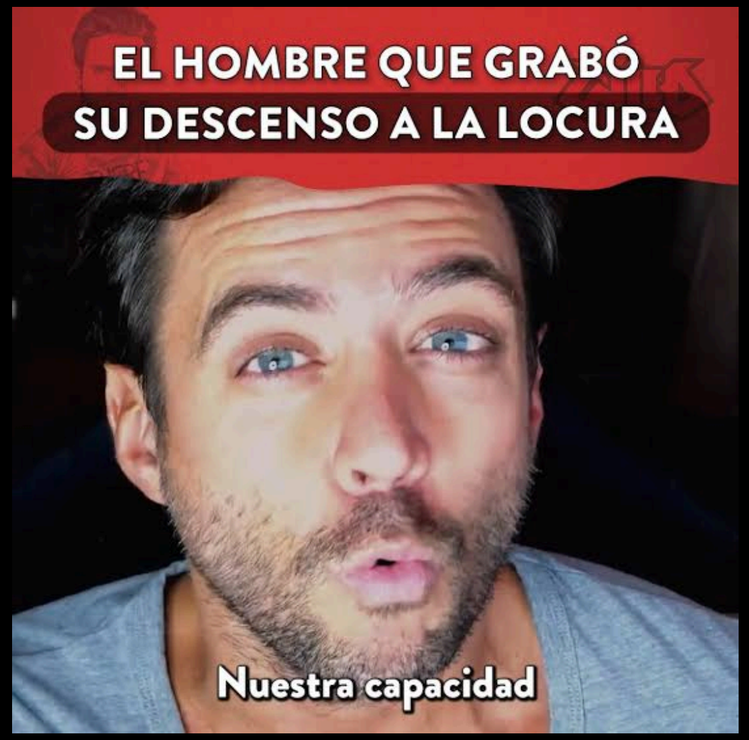


Descenso recursivo

RECORDEMOS MICRO



- Para el primer proyecto de este curso, se desarrolló un compilador completo para el lenguaje MICRO
- La gramática de MICRO era LL(1)... pero no se usó ninguna tabla
- Se usó una técnica de compilación conocida como “Descenso recursivo”
- Equivalente a LL(1)
- Si una gramática es LL(1) se puede hacer por descenso recursivo y viceversa
- Ambas técnicas tienen los mismos requisitos



Descenso recursivo



- Básicamente se toma la gramática y esta se convierte en código
- Siempre hay un token actual de la hilera que se está parseando
- Cada no terminal tendrá un procedimiento asociado
- Dentro del procedimiento de cada no terminal se escoge cuál regla de la gramática aplicar, dependiendo del token actual.
- El lado derecho de cada regla de la gramática se convierte en llamadas a los procesos de cada no terminal o un match del símbolo actual con el terminal esperado.



Descenso recursivo

EJEMPLO 1

Para la gramática:

1. E → ; E’
2. E’ → + ; E’ | ε

El código quedaría así:

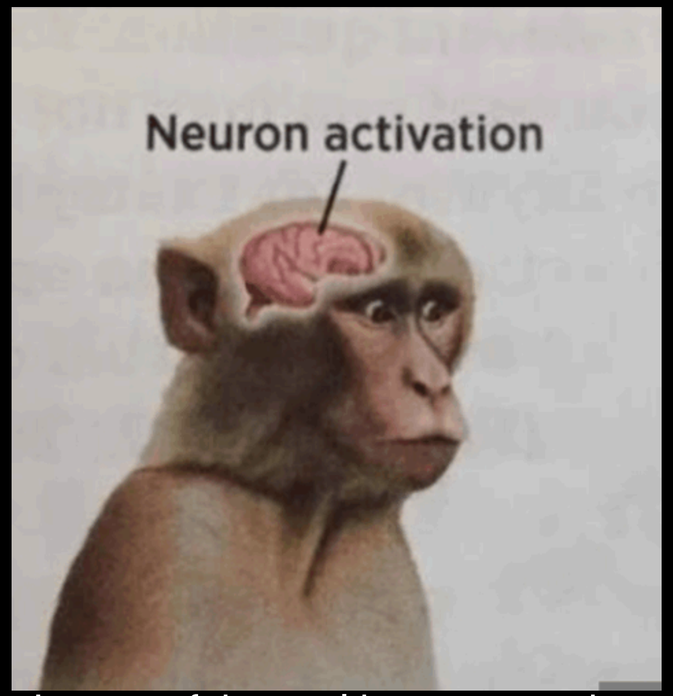
```
void E(){
    match(';');
    E_prima();
}
void E_prima(){
    if (c=='+'){
        match('+');
        match(';');
        E_prima();
    }
    else{
        return;
    }
}
```




```
void match(char t){
    if ( t == c )
        c = gettoken();
    else
        printf("ERROR");
}

void main(){
    c = gettoken();
    E();
}
```

Descenso recursivo



Los que fuimos el jueves cuando Torres explicó el tema

EJEMPLO 2

Para la gramática:

1. SQ → S SQ’
2. SQ’ → ; SQ | ε
3. S → <statement>

El código quedaría así:

```
void SQ(){
    S();
    SQ_Prima();
}

void SQ_Prima(){
    if(c==';'){
        match(';');
        SQ();
    }
    else
        return;
}
```

```
void S(){
    match('<statement>');
}

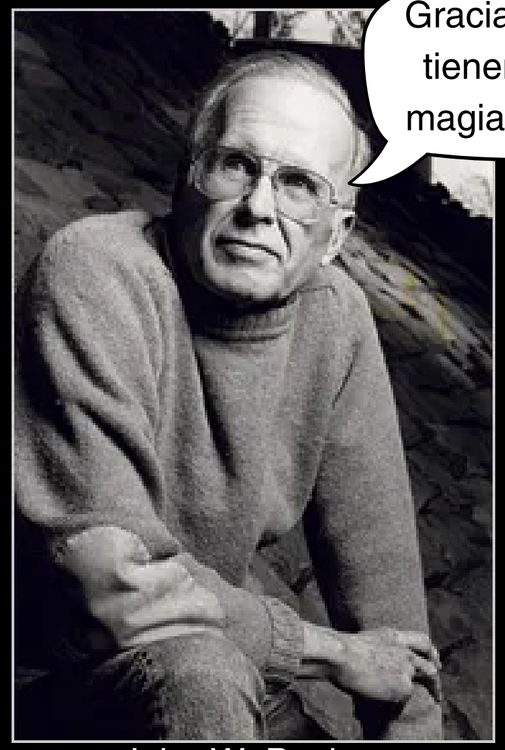
void match(char t){
    if ( t == c )
        c = gettoken();
    else
        printf("ERROR");
}

void main(){
    c = gettoken();
    SQ();
}
```



Los que no fueron

Descenso recursivo



John W. Backus

EJEMPLO 3

Para la gramática:

1. S → IFS | <other>
2. IFS → if (EX) S EL
3. EL → else S | ε
4. EX → 0 | 1

El código quedaría así:

```
void S(){
    if (c=='if')
        IFS();
    else
        match('<other>');
}

void IFS(){
    match('if');
    match('(');
    EX();
    match(')');
    S();
    EL();
}
```

```
void EL(){
    if (c=='else'){
        match('else');
        S();
    }
    else
        return;
}

EX(){
    if(c=='0')
        match('0');
    else
        match('1');
}
```

```
void match(char t){
    if ( t == c )
        c = gettoken();
    else
        printf("ERROR");
}

void main(){
    c = gettoken();
    S();
}
```

Errores y LL(1)

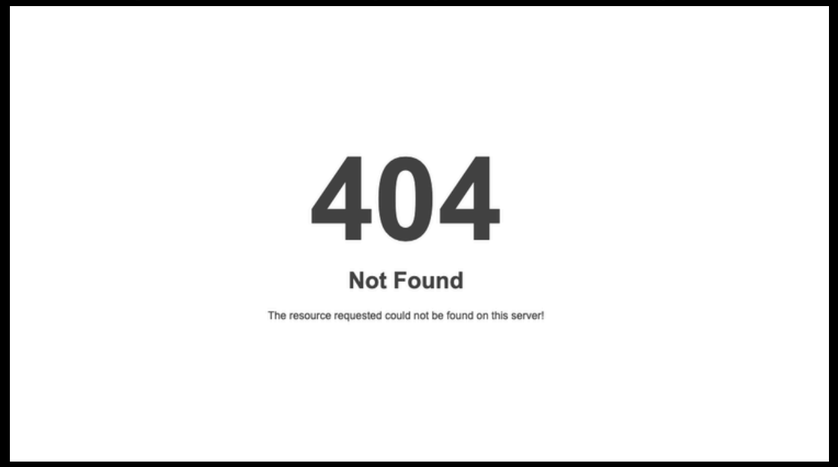


- El algoritmo detecta hileras inválidas - entrada vacía en la tabla LL(1)
- Estrategia simplista:
  - “error de sintaxis aquí”
  - No revisar el resto de la hilera :b
- Mejor enfoque:
  - Mensajes de error: dar el mejor diagnóstico posible de la causa del error
  - Recuperación de error: volver a sincronizar el autómata para reportar otros errores en la hilera



Mensajes de error

El mayor machetazo por parte de los ingenieros:



- Es casi un arte
- Distintos compiladores dan mensajes diferentes para el mismo error
- Hay muchas causas posibles
- Es particular para cada lenguaje y situación
- A veces se hacen de manera incremental:
  - Dar solo el número de error
  - Ver ejemplos del error e ir mejorando la redacción

```
    char D[] = "HelloWorld"
}
```

Mensajes de error

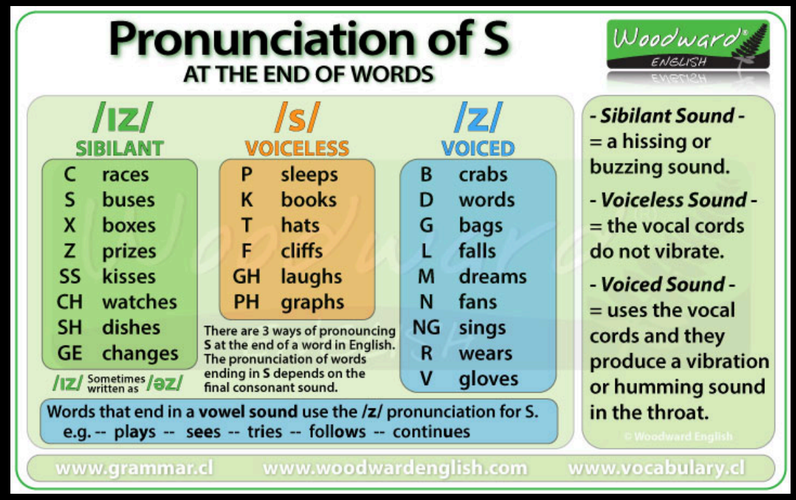
EJEMPLO



- Consideramos la siguiente gramática:

1. S → a
2. S → (SR
3. R → , SR
4. R →)

- Genera “S-expressions”, o sea, hileras de la forma:
  - a
  - (a)
  - (a,a,a)
  - (a,(a,a))
  - ((a,(a,a),(a,a)),a)



Mensajes de error

EJEMPLO

- Armamos la tabla de parsing, la cual queda de la siguiente manera

	a	**,**	**(**	**)**	**$**
S	1		2		
R		3		4	
$					:D

Mensajes de error



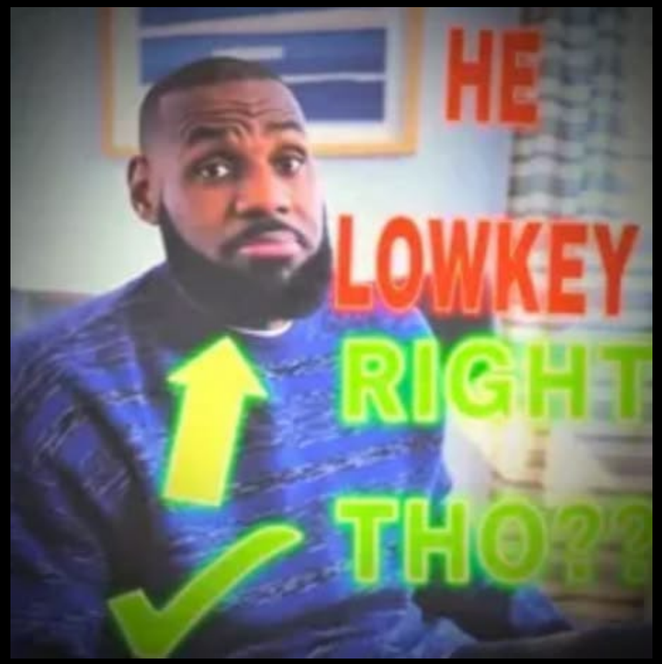
EJEMPLO

- Todos los mensajes podrían ser de la forma: “Aparece ____ cuando se esperaba ____”.
- Calza bien para el caso del match
- Puede ser confuso para no terminal y un terminal
- Analicemos las entradas vacías:

	a	**,**	**(**	**)**	**$**
S	1	*a*	2	*b*	*c*
R	*d*	3	*e*	4	*f*
$	*g*	*h*	*i*	*j*	:D

Mensajes de error

EJEMPLO

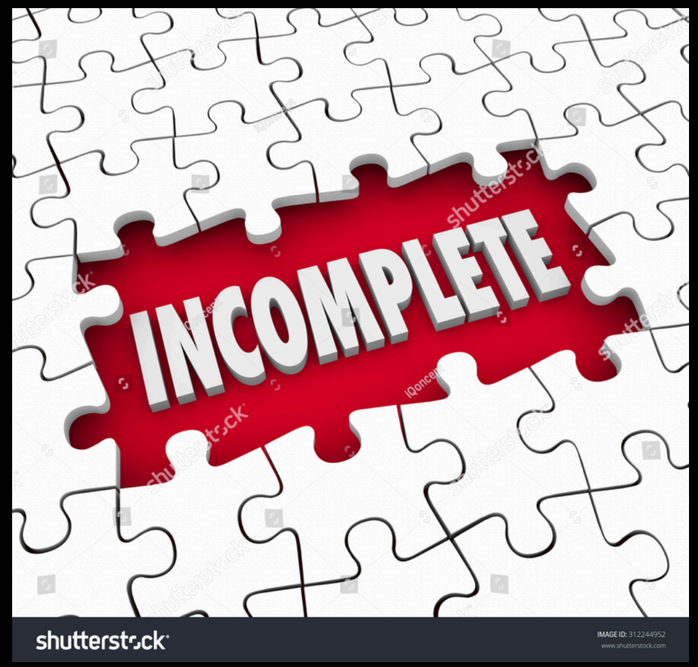


- En los errores *a* y *b* se esperaba un “S” (S-expression) y esos inicios son inválidos
- Un mensaje apropiado sería:
  “Aparece ‘,’ cuando se esperaba S-expression”
  y
  “Aparece ‘)’ cuando se esperaba S-expression”

	a	**,**	**(**	**)**	**$**
S	1	*a*	2	*b*	*c*
R	*d*	3	*e*	4	*f*
$	*g*	*h*	*i*	*j*	:D

Mensajes de error

EJEMPLO



- El error *c* es diferente al *a* y *b*
- Las hileras que lo generan son de la forma:
  - ($
  - (a,$
  - $
- Un mensaje apropiado sería: “S-expression incompleta”

	a	**,**	**(**	**)**	**$**
S	1	*a*	2	*b*	*c*
R	*d*	3	*e*	4	*f*
$	*g*	*h*	*i*	*j*	:D

Mensajes de error

EJEMPLO

Imágen ofensiva

- Los errores *d*, *e* y *f* están asociados al no terminal R
- R no tiene un significado evidente para los programadores
- Podríamos mencionar las cosas que esperábamos cuando llegó el token ofensivo:

“ ‘a’ aparece cuando se esperaba ‘,’ o ‘)’ ”
“ ‘(’ aparece cuando se esperaba ‘,’ o ‘)’ ”
“ Fin de la hilera aparece cuando se esperaba ‘,’ o ‘)’ ”

	a	**,**	**(**	**)**	**$**
S	1	*a*	2	*b*	*c*
R	*d*	3	*e*	4	*f*
$	*g*	*h*	*i*	*j*	:D

Mensajes de error

EJEMPLO



- Las hileras que típicamente causan los errores *d* y *e* son de la forma
  - (a a)$
  - ((a) a)$
  - (a (a))$
- Un mejor mensaje de error para esos casos es:

“ Falta ‘,’ ”

	a	,	(	)	$
S	1	*a*	2	*b*	*c*
R	*d*	3	*e*	4	*f*
$	*g*	*h*	*i*	*j*	:D

Mensajes de error

EJEMPLO



- Las hileras que típicamente causan el error *f* son de la forma:
  - (a,a$
  - ((a),a$
  - (a$
- Un mejor mensaje de error para este caso sería:

“ Falta ‘)’ ”

	a	**,**	**(**	**)**	**$**
S	1	*a*	2	*b*	*c*
R	*d*	3	*e*	4	*f*
$	*g*	*h*	*i*	*j*	:D

Mensajes de error

EJEMPLO



- Los errores *g*, *h* y *j* se dan cuando el autómata cree que ya la hilera debe terminar y aún hay símbolos:
  - (a) a $
  - ((a),a),a) $
  - (a)(a)) $
- Un mensaje de error apropiado para estos casos sería:

"____ aparece después de fin de S-expression"

	a	**,**	**(**	**)**	**$**
S	1	*a*	2	*b*	*c*
R	*d*	3	*e*	4	*f*
$	*g*	*h*	*i*	*j*	:D

Mensajes de error

EJEMPLO

- Se puede construir una tabla de mensajes de error para cada situación:

Error	Mensaje
a	Aparece ‘,’ cuando se esperaba S-expression
b	Aparece ‘)’ cuando se esperaba S-expression
c	S-expression incompleta
d	Falta ‘,’
e	Falta ‘,’
f	Falta ‘)’
g	‘a’ aparece después de fin de S-expression
h	‘,’ aparece después de fin de S-expression
i	‘(’ aparece después de fin de S-expression
j	‘)’ aparece después de fin de S-expression



STEVEN LOAÍCIGA SEGURA

5 DE NOVIEMBRE DE 2026



SIGUIENTE CLASE: MARTES 10 DE NOVIEMBRE

COMPILADORES E INTÉRPRETES

YOU MUST LISTEN TO THE DARK SIDE OF THE MOON